

Lectures on Monte Carlo Theory

Chapter 2: The Theory of Generators

Based on Leskelä & Vihola

Chapter 2 Outline

- 1 2.1 Introduction
- 2 2.2 Random Permutations
- 3 2.3 Pseudorandom Number Generators in Python
- 4 2.4 Simple Generators: Historical Perspective
- 5 2.5 Testing Good Properties of PRNGs
- 6 2.6 Bibliographical Notes

2.1 Introduction I

Why Generators?

Randomness is a cornerstone of Monte Carlo methods. In practice, we need *sequences of random numbers* that behave as if they were truly random.

Key applications:

- Simulating random processes (stochastic models)
- Monte Carlo integration
- Cryptographic systems
- Statistical sampling and resampling

Historical curiosity – Middle-Square Method (von Neumann, 1940s):

- Start with a 4-digit seed, square it, extract middle 4 digits
- Example: $43^2 = 1849$, $84^2 = 7056$, $05^2 = 0025$, $02^2 = 0004$, $00^2 = 0000$
- Problem: tends to cycle into short loops (e.g., $\dots \rightarrow 0000 \rightarrow 0000$)

2.1 Introduction II

Definition 2.1.1 – Pseudorandom Number Generator (PRNG)

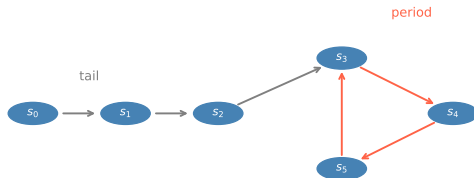
A PRNG is a 5-tuple (S, s_0, f, V, g) where:

- S is a finite *state space*,
- $s_0 \in S$ is the initial state (the *seed*),
- $f : S \rightarrow S$ is the *transition function*: $s_{n+1} = f(s_n)$,
- V is a finite set of output values,
- $g : S \rightarrow V$ maps states to outputs: $u_i = g(s_i)$, $i = 0, 1, \dots$

Since S is finite, the sequence *eventually cycles*. The cycle length is the **period**. Longer periods are better.

2.1 Introduction III

PRNG state sequence: tail + periodic cycle



The period cannot exceed $|S|$.

2.1.1 Formal Definition: Random Numbers I

Three types of random sequences used in Monte Carlo:

Sequence of random numbers from $[0, 1)$

$U_1, U_2, \dots$ where each $U_j \sim \mathcal{U}[0, 1)$, i.e.,

$$\mathbb{P}(U_j \leq t) = \begin{cases} 0 & t < 0, \\ t & 0 \leq t < 1, \\ 1 & t \geq 1, \end{cases}$$

and $U_1, U_2, \dots$ are i.i.d.:

$$\mathbb{P}(U_1 \leq t_1, \dots, U_n \leq t_n) = t_1 t_2 \cdots t_n, \quad 0 \leq t_i \leq 1.$$

2.1.1 Formal Definition: Random Numbers II

Sequence of random bits

$B_1, B_2, \dots$ where $B_j \in \{0, 1\}$, i.i.d., with

$$\mathbb{P}(B_j = 0) = \mathbb{P}(B_j = 1) = \frac{1}{2}, \quad \mathbb{P}(B_1 = b_1, \dots, B_n = b_n) = \frac{1}{2^n}.$$

Sequence of random integers from $\bar{M}$

$Y_1, Y_2, \dots$ where $Y_j \in \{0, 1, \dots, M-1\} =: \bar{M}$, i.i.d., each uniformly distributed on $\bar{M}$.

Notation: We write $U \sim \mathcal{U}(0, 1)$, $B \sim \mathcal{U}(\{0, 1\})$, $Y \sim \mathcal{U}(\bar{M})$.

PRNGs generate *pseudorandom numbers*: deterministic sequences that *mimic* statistical properties of true random sequences.

2.2 Random Permutations I

Goal: Generate a uniformly random permutation of $\{1, \dots, n\}$, i.e., each of the $n!$ arrangements has equal probability $1/n!$.

Algorithm 1: Fisher-Yates (Knuth Shuffle)

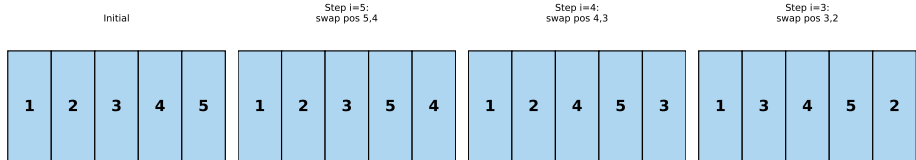
Input: Integer n ; pseudorandom numbers $\mathbf{u} = (u_1, \dots, u_n)$, $u_i \in [0, 1)$

Output: A random permutation of $\{1, \dots, n\}$

- 1 Initialize: $S[i] = i$ for $i = 1, \dots, n$
- 2 **for** $i = n$ **downto** 1 **do**
- 3 $k = 1 + \lfloor iu_i \rfloor$ (pick random position in $\{1, \dots, i\}$)
- 4 Swap($S[k], S[i]$)
- 5 **end for**
- 6 **return** S

2.2 Random Permutations II

Fisher-Yates Algorithm: Shuffling [1, 2, 3, 4, 5]



2.2 Random Permutations III

Algorithm 2: Perm-Sort

An alternative: sort indices by u_i values.

- ① $S[i] = i$ for $i = 1, \dots, n$
- ② Sort indices in S such that $u_{S[1]} \leq u_{S[2]} \leq \dots \leq u_{S[n]}$
- ③ **return** S

Both algorithms produce a uniform random permutation, given a uniform input sequence.

Correctness of Fisher-Yates: At step i , the element placed in position i is chosen uniformly from the remaining i elements. This inductively yields a uniform random permutation.

2.2.2 Shuffling Cards I

Physical analogy: Shuffling a deck of 52 cards.

- 52 cards $\Leftrightarrow n = 52$
- Each arrangement has probability $1/52!$
- $52! \approx 8.07 \times 10^{67}$ possible orderings

Card Shuffling Schemes:

- **Top-To-Random:** Take the top card, insert it at a uniformly random position.
- **Riffle Shuffle:** Split deck into two roughly equal halves; interleave by dropping cards with probability proportional to pile size.
- **Time-Reversed Riffle Shuffle:** The reverse operation.

2.2.2 Shuffling Cards II

How many shuffles to reach uniformity?

After k steps, the distribution of X_k (the deck permutation) is compared to the uniform distribution using the *total variation distance*:

$$d_k = \frac{1}{2} \sum_{\sigma \in \mathcal{S}_n} \left| \mathbb{P}(X_k = \sigma) - \frac{1}{n!} \right|.$$

As $k \rightarrow \infty$, $d_k \rightarrow 0$ (mixing property of Markov chains).

Key result (Bayer & Diaconis): For $n = 52$ cards, approximately **7 riffle shuffles** are needed before the deck is close to random. Fewer shuffles leave detectable structure.

2.3 Pseudorandom Number Generators in Python I

Modern Python (NumPy): The recommended approach uses `numpy.random`.

2.3.1 Python Generator Hierarchy

- `default_rng(seed)` — creates a Generator with PCG64 (default)
- `PCG64(seed)` — Permuted Congruential Generator (64-bit)
- `MT19937(seed)` — Mersenne Twister (legacy, period $2^{19937} - 1$)
- `Philox`, `SFC64` — alternative high-quality generators

PCG64 properties:

- Period: 2^{128} (essentially unlimited)
- Passes all standard statistical tests
- Multiple independent streams possible
- Recommended for Monte Carlo simulations

2.3 Pseudorandom Number Generators in Python II

MT19937 (Mersenne Twister):

- Period: $2^{19937} - 1$ (extremely long)
- State space: 624 32-bit integers
- Widely used historically (Python's `random` module, older NumPy)
- Fails some modern statistical tests
- Not suitable for cryptographic use

Getting random numbers from $\{0, \dots, M - 1\}$: Given $u_i \in [0, 1)$, compute $y_i = \lfloor Mu_i \rfloor$.

2.3.1 Python Code: Using PRNGs

```
import numpy as np

# Create a generator (PCG64 is the default)
rng = np.random.default_rng(seed=42)

# Generate n=10 uniform random numbers in [0, 1)
u = rng.random(10)
print("Uniform [0,1):", u)

# Generate integers in {0, 1, ..., 9}
y = rng.integers(0, 10, size=10)
print("Integers:", y)

# Fisher-Yates shuffle of [0,...,n-1]
n = 52
perm = rng.permutation(n)
print("Permutation:", perm[:10], "...")

# Explicitly use PCG64 or MT19937
from numpy.random import PCG64, MT19937, Generator
rng_pcg = Generator(PCG64(seed=0))
rng_mt = Generator(MT19937(seed=0))

# Compare outputs
print("PCG64:", rng_pcg.random(5))
print("MT19937:", rng_mt.random(5))
```

2.3.2 Cryptographic PRNGs: RC4 I

Cryptographic PRNGs must satisfy:

- All standard statistical tests (like non-cryptographic PRNGs)
- **In addition:** computationally infeasible to predict future/past outputs given partial observations

Definition: A PRNG is *cryptographically secure* (CSPRNG) if no polynomial-time algorithm can distinguish its output from a truly random sequence with non-negligible advantage.

RC4 (Rivest Cipher 4):

- Stream cipher designed in 1987 by Ron Rivest (RSA Security)
- Based on a *key-scheduling algorithm* (KSA) and a *pseudo-random generation algorithm* (PRGA)
- Was widely used in TLS/SSL, WEP, PDF encryption
- Now considered *insecure* — deprecated in TLS 1.3

2.3.2 Cryptographic PRNGs: RC4 II

RC4: Key-Scheduling Algorithm (KSA)

Input: n , key $K[0], \dots, K[L-1]$

Output: Permutation $S[0], \dots, S[n-1]$

- 1 Initialize: $S[i] = i$ for $i = 0, \dots, n-1$
- 2 $j := 0$
- 3 **for** $i = 0$ to $n-1$ **do:** $j = (j + S[i] + K[i \bmod L]) \bmod n$; $\text{Swap}(S[i], S[j])$

RC4: Pseudo-Random Generation Algorithm (PRGA)

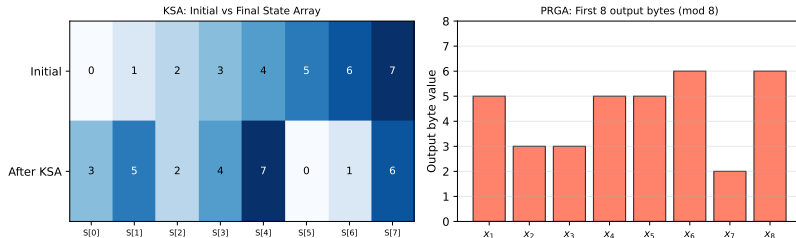
Input: $S[0], \dots, S[n-1]$ (from KSA)

Output: Bytes $x_1, x_2, \dots$

- $i := 0, j := 0$
- **while** generating output **do:** $i = (i + 1) \bmod n$; $j = (j + S[i]) \bmod n$; $\text{Swap}(S[i], S[j])$;
 $x_r = S[(S[i] + S[j]) \bmod n]$

2.3.2 Cryptographic PRNGs: RC4 III

RC4 Algorithm: KSA + PRGA (illustrated with $n = 8$)



Security note: RC4 has known biases in the first few bytes of output and statistical weaknesses discovered by Fluhrer, Mantin, and Shamir (2001). The IETF deprecated it in RFC 7465.

2.3.2 Python Code: RC4 Implementation

```
def rc4_ksa(n, key):
    """Key-Scheduling Algorithm."""
    S = list(range(n))
    j = 0
    L = len(key)
    for i in range(n):
        j = (j + S[i] + key[i % L]) % n
        S[i], S[j] = S[j], S[i]
    return S

def rc4_prga(S, num_bytes):
    """Pseudo-Random Generation Algorithm."""
    n = len(S); i = j = 0; output = []
    while len(output) < num_bytes:
        i = (i + 1) % n
        j = (j + S[i]) % n
        S[i], S[j] = S[j], S[i]
        t = (S[i] + S[j]) % n
        output.append(S[t])
    return output

# Example: 256-element state, key = [3, 1, 4, 1, 5]
key = [3, 1, 4, 1, 5]
S = rc4_ksa(256, key)
stream = rc4_prga(S, 20)
print("RC4 keystream:", stream)
```

2.4 Simple Generators: Historical Perspective I

Overview: Before modern PRNGs, several simple generators were widely used. They are primarily of *historical and educational* value:

- Easy to implement and analyze mathematically
- Typically 32-bit with limited period
- Poor statistical quality by modern standards
- Vulnerable to predictability attacks

Understanding them builds intuition for PRNG design challenges.

2.4.1 Linear Congruential Generators (LCG) I

Linear Congruential Recurrence

$$x_{n+1} = (ax_n + c) \bmod M \quad (2.4.1)$$

Parameters:

- $M > 0$: **modulus**
- $0 \leq a < M$: **multiplier**
- $0 \leq c < M$: **increment**
- $0 \leq x_0 < M$: **seed**

Output: $u_n = x_n/M \in [0, 1)$

The period cannot exceed M . Good choice of parameters gives period exactly M .

2.4.1 Linear Congruential Generators (LCG) II

Theorem 2.4.1 (Full Period Conditions)

Let $b = a - 1$. The sequence (x_n) has period M if and only if:

- c is coprime with M ,
- b is a multiple of p for every prime p dividing M ,
- If 4 divides M , then $a \equiv 1 \pmod{4}$.

Historical examples:

Example 2.4.2 $M = 2^{31} - 1 = 2147483647$, $a = 7^5 = 16807$, $c = 0$.

Period = $M - 1$ (prime modulus). Also: $M = 2^{31} - 1$, $a = 40014$.

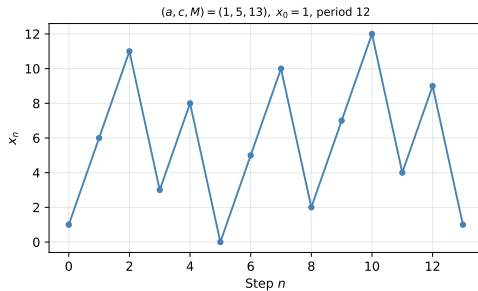
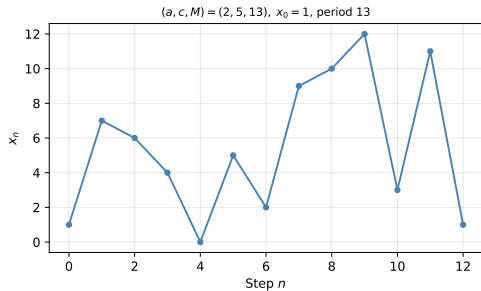
Example 2.4.3 (Resing) $(a, c, M) = (1, 5, 13)$, $x_0 = 1$:

Sequence: 1, 6, 11, 3, 8, 0, 5, 10, 2, 7, 12, 4, 9, 1, ..., period 13.

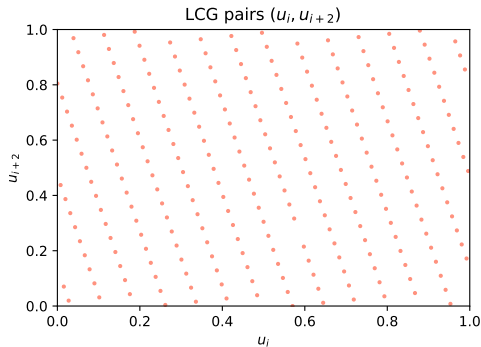
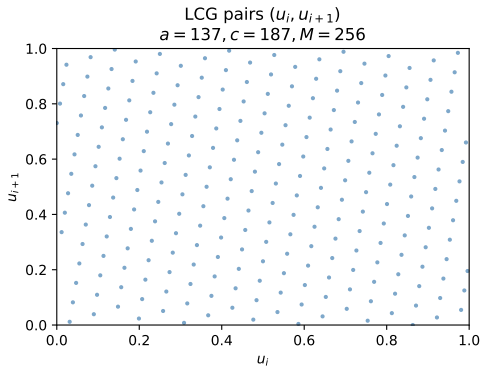
Example 2.4.3 $(a, c, M) = (2, 5, 13)$, $x_0 = 1$:

Sequence: 1, 7, 6, 4, 0, 5, 2, 9, ..., period 12. If $x_0 = 8$: constant sequence (period 1).

2.4.1 Linear Congruential Generators (LCG) III



2.4.1 Linear Congruential Generators (LCG) IV



Note the *lattice structure* in pairs (u_i, u_{i+1}) — a known weakness of LCGs.

2.4.1 Linear Congruential Generators (LCG) V

Commercial LCG parameters:

System	Recurrence	Output
Java	$x_{i+1} = (25214903917 x_i + 11) \bmod 2^{48}$	$u_i = (2^{27} \lfloor x_{2i}/2^{22} \rfloor + \lfloor x_{2i+1}/2^{21} \rfloor)/2^{51}$
Visual Basic	$x_i = (1140671485 x_{i-1} + 12820163) \bmod 2^{24}$	$u_i = x_i/2^{24}$
Excel	$u_i = (0.9821 u_{i-1} + 0.211327) \bmod 1$	—
rand48 (UNIX)	Same as Java for x_i	$u_i = x_i/2^{48}$

These fail modern statistical tests and should *not* be used for serious simulations.

2.4.1 Linear Congruential Generators (LCG) VI

Generalized Linear Congruential Generator:

$$x_n = (a_1x_{n-1} + a_2x_{n-2} + \cdots + a_kx_{n-k} + c) \bmod M.$$

Achieves period up to M^k with appropriate coefficients.

Quadratic Congruential Generator I (QCG I):

$$x_{i+1} = x_i^2 \bmod M,$$

where M is a 512-bit prime.

Quadratic Congruential Generator II (QCG II):

$$x_{i+1} = 2x_i^2 + 3x_i + 1 \bmod 2^{512}.$$

Cubic Congruential Generator (CCG):

$$x_{i+1} = 2x_i^3 \bmod 2^{512}.$$

2.4.1 Linear Congruential Generators (LCG) VII

Fibonacci Generator:

$$x_n = x_{n-1} + x_{n-2} \bmod M \quad (n \geq 2).$$

Lagged Fibonacci (Knuth):

$$x_n = x_{n-k} + x_{n-l} \bmod M, \quad n \geq \max(k, l),$$

with $k = 100$, $l = 37$, $M = 2^{30}$ (Knuth's recommended parameters).

2.4.1 Python Code: LCG Implementation

```
import numpy as np
import matplotlib.pyplot as plt

def lcg(a, c, M, x0, n):
    """Linear Congruential Generator."""
    x = x0
    seq = []
    for _ in range(n):
        x = (a * x + c) % M
        seq.append(x / M)
    return seq

# Example 2.4.3: (a,c,M) = (2,5,13), x0 = 1, period = 12
u1 = lcg(a=2, c=5, M=13, x0=1, n=14)
print("LCG (2,5,13), x0=1:", [round(x,3) for x in u1])

# Compare with PCG64
rng = np.random.default_rng(seed=0)
u_pcg = rng.random(14)
print("PCG64:", [round(x,3) for x in u_pcg])

# Plot pairs to see lattice structure
u = lcg(a=137, c=187, M=256, x0=1, n=256)
plt.figure(figsize=(5,4))
plt.scatter(u[:-1], u[1:], s=4, alpha=0.7)
plt.title('LCG pairs: lattice structure visible')
plt.xlabel(r'$u_{i-1}$'); plt.ylabel(r'$u_i$')
```

2.4.1 L'Ecuyer Mixed Generator I

Idea: Combine two LCGs to get much longer period and better quality.

L'Ecuyer Mixed Generator

- ① $x_n = (A_1 x_{n-2} - A_2 x_{n-3}) \bmod M_1$
- ② $y_n = (B_1 y_{n-1} - B_2 y_{n-3}) \bmod M_2$
- ③ $z_n = (x_n - y_n) \bmod M_1$
- ④ $u_n = z_n / (M_1 + 1)$ if $z_n > 0$; otherwise $u_n = M_1 / (M_1 + 1)$

Parameters: $M_1 = 4294967087$, $M_2 = 4294944443$,
 $A_1 = 1403580$, $A_2 = 810728$, $B_1 = 527612$, $B_2 = 1370589$.

Period $\approx 2^{191}$, passing many statistical tests. This was state-of-the-art before PCG64/MT19937.

2.4.2 Bit Generators I

Sequences of $\{0, 1\}$ -valued pseudorandom numbers are **pseudorandom bits**.

Register operations:

- **XOR** ($\oplus$): $p \oplus q = 1$ iff an odd number of inputs is 1.

$$\mathbf{p} \oplus \mathbf{q} = (p_1 \oplus q_1, \dots, p_n \oplus q_n).$$

- **Bit shifts:** $\ll$ (left shift), $\gg$ (right shift). For example, $x = y \ll 2$ shifts y left by 2 positions (fills with zeros).

XOR truth table:

p	q	$p \oplus q$
0	0	0
0	1	1
1	0	1
1	1	0

2.4.2 Bit Generators II

XORG: Exclusive OR Generator

$$x_i = x_{i-1} \oplus x_{i-127}, \quad i \geq 128,$$

initialized with a 127-bit seed. Simple and fast.

LFSR: Linear Feedback Shift Register

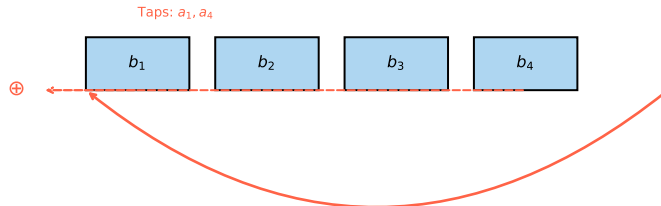
$$x_i = (a_1 x_{i-1} \oplus a_2 x_{i-2} \oplus \cdots \oplus a_k x_{i-k}),$$

where $a_j \in \{0, 1\}$ are **feedback taps**. With appropriate taps, achieves maximum period $2^k - 1$.

2.4.2 Bit Generators III

Linear Feedback Shift Register (LFSR)

$$x_i = (a_1 x_{i-1} \oplus a_2 x_{i-2} \oplus \cdots \oplus a_k x_{i-k})$$



2.4.2 Bit Generators IV

Blum-Blum-Shub (BBS) Generator

$$x_{i+1} = x_i^2 \bmod M, \quad M = pq \text{ (product of two large primes).}$$

Output: the least significant bit of x_i .

Cryptographically secure under the quadratic residuosity assumption, but slow for Monte Carlo use.

Shift-XOR (SXR) Generator

- ➊ Seed $x_0 \in \{0, 1, \dots, 2^n - 1\}$ ($n = 32$ or $n = 64$)
- ➋ $x_{i+1} = (x_i \oplus (x_i \ll a)) \oplus (x_i \gg b)$
- ➌ Output x_{i+1}

For $n = 32$: $a = 13$, $b = 17$ gives a simple efficient generator.

Example: $x_0 = 0xABCDE123$:

$$x_1 = (x_0 \oplus (x_0 \ll 13)) \oplus (x_0 \gg 17), \quad \dots$$

2.4.2 Bit Generators V

Properties: (i) simple bitwise implementation; (ii) short period; (iii) good randomness for simple uses but fails advanced tests.

2.5 Testing Good Properties of PRNGs I

Two types of tests:

- 1 **Theoretical tests:** examine internal generator structure (e.g., lattice tests, spectral tests). Detect structural weaknesses without running the generator.
- 2 **Statistical tests:** treat the PRNG as a black box; run it and apply hypothesis tests to the output.

Test batteries:

- **Diehard** tests (Marsaglia): 18 tests; widely used historically
- **TestU01** (L'Ecuyer & Simard): comprehensive suite; small/medium/big crush
- **NIST SP 800-22** (National Institute of Standards and Technology): 15 tests for cryptographic applications
- **Uniformity, Scalability, Consistency:** basic requirements for any good PRNG

Motivating experiment: Generate 256 pairs (x_i, x_{i+1}) for the LCG $x_j = (137x_{j-1} + 187) \bmod 256$. Plot the pairs and triples. The lattice structure is immediately visible (see LCG pairs figure above).

2.5.1 Preliminary Remarks I

Notation:

- $U \sim \mathcal{U}(0, 1)$: uniform on $[0, 1)$
- $B \sim \mathcal{U}(\{0, 1\})$: uniform random bit
- $Y \sim \mathcal{U}(\bar{M})$: uniform on $\{0, \dots, M - 1\}$

Converting Between Formats (2.5.1)

From $[0, 1)$ to $\bar{M}$:

$$y_i = \lfloor M u_i \rfloor.$$

From $\bar{M}$ to $[0, 1)$:

$$u_i = y_i / M.$$

From $[0, 1)$ to bits (careful method): First convert $u_i \rightarrow y_i \in \bar{M}$ (with $M = 2^k$), then use binary expansion of y_i .

Warning: The naive conversion $b_i = \mathbb{K}[u_i \geq 0.5]$ discards all but the most significant bit, and can be exploited to cheat tests.

2.5.1 Preliminary Remarks II

Grouping into Multidimensional Vectors (2.5.1.2)

Given $n = mr$ numbers $u_1, \dots, u_n$, group into r vectors of length m :

$$\mathbf{u}_1 = (u_1, \dots, u_m), \quad \mathbf{u}_2 = (u_{m+1}, \dots, u_{2m}), \quad \dots, \quad \mathbf{u}_r = (u_{(r-1)m+1}, \dots, u_{rm}).$$

Each $\mathbf{u}_i \in [0, 1)^m$. Similarly for integer sequences: $\mathbf{y}_i \in \bar{M}^m$.

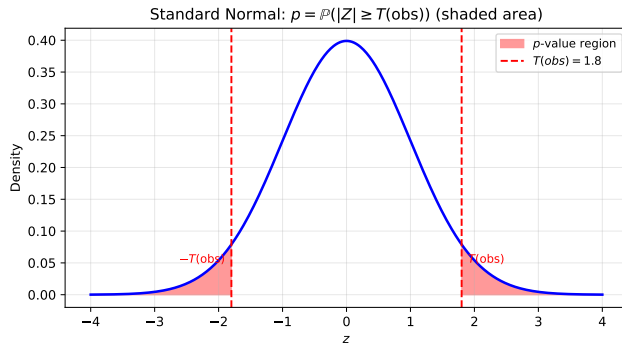
The p-value concept (2.5.1.3)

$$p = \mathbb{P}(T \geq T(\text{obs})) = 1 - F(T(\text{obs})),$$

where T is the test statistic and F is its c.d.f. under $\mathcal{H}_0$.

- Under $\mathcal{H}_0$: $p \sim \mathcal{U}(0, 1)$
- Small p (e.g., < 0.01): evidence *against* randomness
- Large p (e.g., > 0.99): suspiciously “too perfect” (quasi-random)

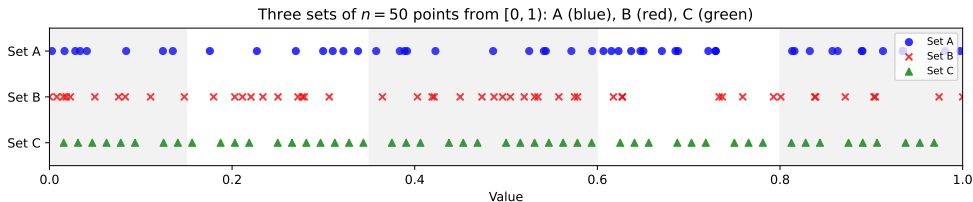
2.5.1 Preliminary Remarks III



2.5.2 Kolmogorov-Smirnov and Chi-Square Tests I

Running example: Three sets of $n = 50$ points from $[0, 1)$:

- **Set A:** PCG64 (truly pseudorandom)
- **Set B:** Nonlinear transform of A (not uniform)
- **Set C:** Quasi-random (van der Corput / Halton sequence)



2.5.2 Kolmogorov-Smirnov and Chi-Square Tests II

Kolmogorov-Smirnov (KS) Test

Empirical CDF:

$$\hat{F}_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(U_i \leq t).$$

KS statistic:

$$D_n = \sup_{0 \leq t \leq 1} |\hat{F}_n(t) - t|.$$

Under $\mathcal{H}_0$ ($U_i \sim \mathcal{U}(0, 1)$, i.i.d.): $D_n \rightarrow 0$ a.s. (Glivenko-Cantelli).

Moreover, $\sqrt{n}D_n$ converges to the **Kolmogorov distribution**:

$$K(t) = \lim_{n \rightarrow \infty} \mathbb{P}(\sqrt{n}D_n \leq t) = 1 - 2J(\exp(-2t^2)), \quad t > 0,$$

where $J(t) = \sum_{j=1}^{\infty} (-1)^{j-1} \exp(-2j^2t^2)$.

Critical values: $\lambda_{0.1} = 1.224$, $\lambda_{0.05} = 1.358$, $\lambda_{0.01} = 1.628$.

2.5.2 Kolmogorov-Smirnov and Chi-Square Tests III

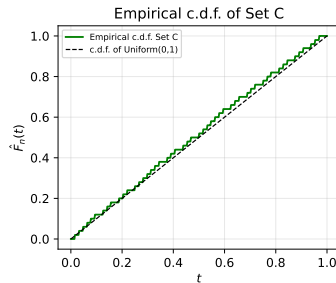
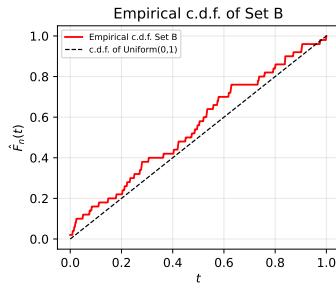
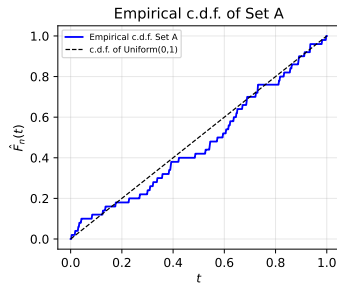
Computing D_n from order statistics: Let $U_{(1)} \leq \dots \leq U_{(n)}$ be the sorted sample. Then:

$$D_n^+ = \max_{1 \leq i \leq n} \left(\frac{i}{n} - U_{(i)} \right), \quad D_n^- = \max_{1 \leq i \leq n} \left(U_{(i)} - \frac{i-1}{n} \right), \quad D_n = \max(D_n^+, D_n^-).$$

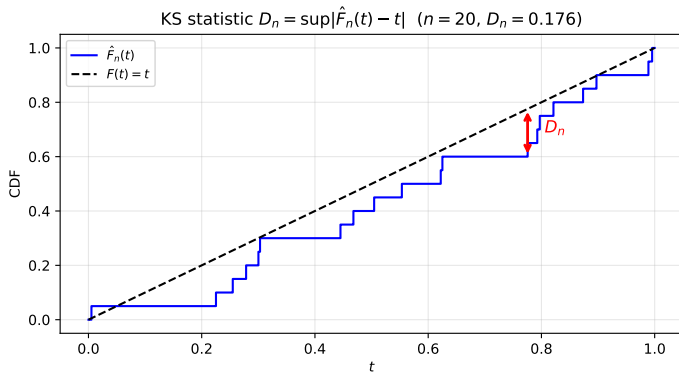
p-value formula (with correction for finite n):

$$p = 1 - K \left(\sqrt{n} D_n(\text{obs}) + \frac{1}{6\sqrt{n}} + \frac{\sqrt{n} D_n(\text{obs}) - 1}{4n} \right).$$

2.5.2 Kolmogorov-Smirnov and Chi-Square Tests IV



2.5.2 Kolmogorov-Smirnov and Chi-Square Tests V



KS test results for Sets A, B, C (Example 2.5.1, $n = 50$):

2.5.2 Kolmogorov-Smirnov and Chi-Square Tests VI

	Set A	Set B	Set C
$D_n(\text{obs})$	0.1192	0.2266	0.0462
p -value	44.17%	0.99%	99.97%
Conclusion	Not rejected	Rejected	Not rejected

Set C (quasi-random) has suspiciously *large* p -value: “too uniform.” Second-level testing detects this (see Sect. 2.5.6).

2.5.2 Kolmogorov-Smirnov and Chi-Square Tests VII

Chi-Square Goodness-of-Fit Test

Given k classes with probabilities $p_1, \dots, p_k$ and r trials, let O_i = observed count in class i , $E_i = rp_i$ = expected count.

$$X_k^2 = \sum_{i=1}^k \frac{(O_i - rp_i)^2}{rp_i} = \sum_{i=1}^k \frac{O_i^2}{rp_i} - r.$$

As $r \rightarrow \infty$: $X_k^2 \xrightarrow{\mathcal{D}} \chi_{k-1}^2$.

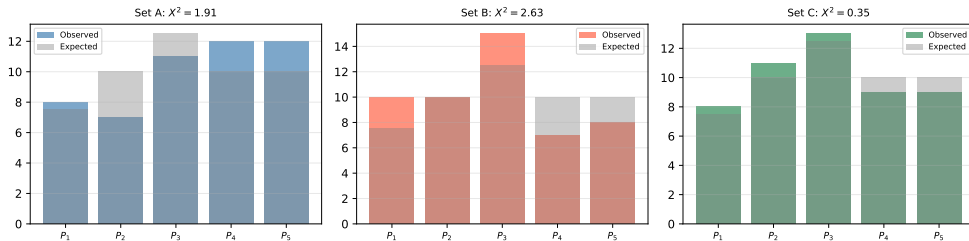
Rule of thumb: Each class needs at least 5 expected observations: $r \geq \lceil 5 / \min_i p_i \rceil$.

Chi-square results for Sets A, B, C (partition $\mathcal{P}$ into 5 bins, $r = 50$):

	Set A	Set B	Set C
$X^2(\text{obs})$	7.32	16.25	0.24
$p\text{-value } (\chi_4^2)$	11.99%	0.27%	99.35%

2.5.2 Kolmogorov-Smirnov and Chi-Square Tests VIII

Chi-Square Test Observed vs Expected Counts ($n = 50$, $k = 5$ bins)



2.5.2 Python Code: KS and Chi-Square Tests

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(0)
n = 50
A = rng.random(n)

# Set B: exponential transform (non-uniform)
eps = 1e-7
B_raw = np.exp(A - 1) - 1 - eps
B = (B_raw - B_raw.min()) / (B_raw.max() - B_raw.min())

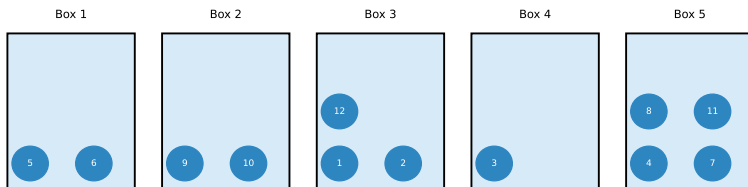
# Kolmogorov-Smirnov test
ks_stat_A, p_A = stats.kstest(A, 'uniform')
ks_stat_B, p_B = stats.kstest(B, 'uniform')
print(f"Set A: D={ks_stat_A:.4f}, p={p_A:.4f}")
print(f"Set B: D={ks_stat_B:.4f}, p={p_B:.4f}")

# Chi-square test with 5 equal bins
bins = [0, 0.2, 0.4, 0.6, 0.8, 1.0]
obs_A, _ = np.histogram(A, bins=bins)
exp = [n/5] * 5
chi2_A, p_chi2_A = stats.chisquare(obs_A, exp)
obs_B, _ = np.histogram(B, bins=bins)
chi2_B, p_chi2_B = stats.chisquare(obs_B, exp)
print(f"Set A chi2: X2={chi2_A:.3f}, p={p_chi2_A:.4f}")
print(f"Set B chi2: X2={chi2_B:.3f}, p={p_chi2_B:.4f}")
```

2.5.3 Tests Based on Balls-and-Boxes Schemes I

General framework: Place r balls into k boxes uniformly at random. Under $\mathcal{H}_0$: each ball independently goes to box i with probability $p_i = 1/k$.

Balls-and-Boxes: $r = 12$ balls placed into $k = 5$ boxes



2.5.3 Tests Based on Balls-and-Boxes Schemes II

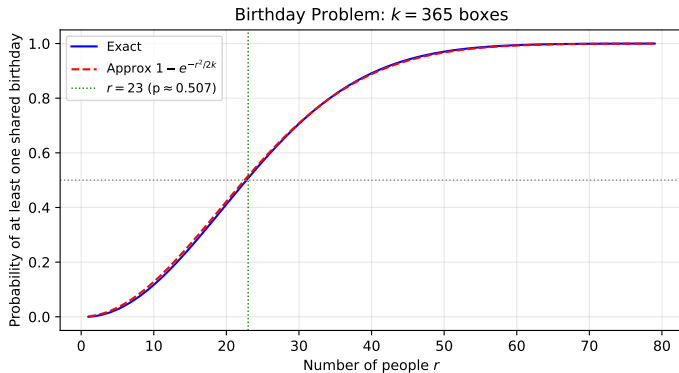
Birthday problem: r people, $k = 365$ boxes (days).

$$p = 1 - \frac{(365)_r}{365^r} = 1 - \prod_{j=1}^{r-1} \left(1 - \frac{j}{365}\right).$$

With $r = 23$ people: $p \approx 0.507$. General approximation:

$$p_{k,r} \approx 1 - \exp\left(-\frac{r^2}{2k}\right). \quad (2.5.12)$$

2.5.3 Tests Based on Balls-and-Boxes Schemes III



2.5.3 Tests Based on Balls-and-Boxes Schemes IV

Converting a sequence to balls-and-boxes:

Given $u_1, \dots, u_n$ from $[0, 1)$, group into r vectors of length m :

$$\mathbf{u}_i = (u_{(i-1)m+1}, \dots, u_{im}), \quad i = 1, \dots, r.$$

Divide $[0, 1)^m$ into $k = L^m$ sub-hypercubes:

$$A_{q_1, \dots, q_m} = \left\{ (v_1, \dots, v_m) : v_i \in \left[\frac{q_i}{L}, \frac{q_i + 1}{L} \right) \right\}, \quad q_i \in \{0, \dots, L-1\}.$$

$\mathbf{u}_i$ falls into box $(q_1, \dots, q_m)$: $y_j = \lfloor Lu_j \rfloor$.

2.5.3 Tests Based on Balls-and-Boxes Schemes V

Collision Test

Count $C_{k,r}$ = number of collisions (balls landing in occupied boxes).

$$\mathbb{P}(C_{k,r} = c) = \frac{k(k-1) \cdots (k-r+c+1) S_2(r, r-c)}{k^r}, \quad (2.5.13)$$

where $S_2(r, s)$ = Stirling number of the second kind.

Poisson approximation: As $r, k \rightarrow \infty$ with $r^2/(2k) \rightarrow \lambda$:

$$\mathbb{P}(C_{k,r} = c) \rightarrow \frac{\lambda^c}{c!} e^{-\lambda}. \quad (2.5.14)$$

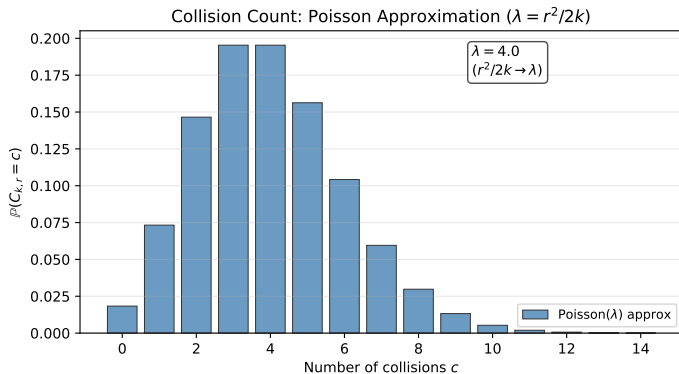
2.5.3 Tests Based on Balls-and-Boxes Schemes VI

Expected Number of Collisions

$$\mathbb{E}C_{k,r} = r - k \left(1 - \left(1 - \frac{1}{k} \right)^r \right) \approx \frac{r^2}{2k}. \quad (2.5.17)$$

For $r = 2^{14}$ and $k = 2^{20}$: $\mathbb{E}C_{k,r} \approx 128$.

2.5.3 Tests Based on Balls-and-Boxes Schemes VII



2.5.3 Tests Based on Balls-and-Boxes Schemes VIII

Spacing Test: Consider spacings between occurrences of $U_j \in (\alpha, \beta]$.

- $\delta = \beta - \alpha$; consecutive spacings $C_1, C_2, \dots$ are i.i.d. $\text{Geometric}(\delta)$
- Group into boxes $A_0 = \{0\}$, $A_1 = \{1\}$, $\dots$, $A_{s-1} = \{s-1\}$, $A_s = \{s, s+1, \dots\}$

Birthday Spacing Test (Marsaglia):

- Split Rn bits into R subsequences of mr bits each
- For each: group into r 25-tuples, treat as birthdays in $k = 2^{25}$ days
- Compute K = number of equal spacings
- Apply chi-square test on K values

K approximately $\text{Poisson}(\lambda)$ with $\lambda = r^3/(4k)$.

2.5.3 Tests Based on Balls-and-Boxes Schemes IX

Poker Test: Group numbers into r 5-tuples $\mathbf{y}_i$. Classify each 5-tuple by the number of distinct values (“poker rank”):

Distinct values	Interpretation
5	All different (like all different suits)
4	One pair
3	Two pairs or three of a kind
2	Full house or four of a kind
1	Five of a kind

Probability of s distinct values:

$$p_s = \frac{M(M-1) \cdots (M-s+1) S_2(5, s)}{M^5},$$

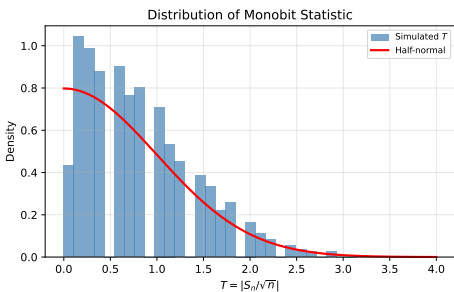
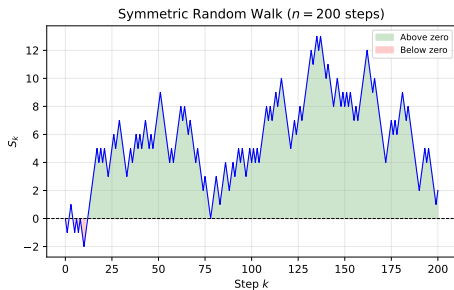
where $S_2(5, 1) = 1$, $S_2(5, 2) = 15$, $S_2(5, 3) = 25$, $S_2(5, 4) = 10$, $S_2(5, 5) = 1$.
Apply chi-square test with 4 degrees of freedom.

2.5.4 Tests Based on Random Walks I

Setup: Let $b_1, b_2, \dots$ be a sequence of bits. Define:

$$X_i = 2B_i - 1 \in \{-1, +1\}, \quad S_0 = 0, \quad S_k = \sum_{i=1}^k X_i.$$

(S_k) is a *symmetric random walk*. Under $\mathcal{H}_0$: $\mathbb{E}X_i = 0$, $\text{Var}X_i = 1$.



2.5.4 Tests Based on Random Walks II

Frequency (Monobit) Test

By the CLT: $S_n/\sqrt{n} \xrightarrow{\mathcal{D}} \mathcal{N}(0, 1)$.

Test statistic: $T(\text{obs}) = |S_n/\sqrt{n}|$ follows a half-normal distribution.

$$p = \text{erfc}\left(\frac{T(\text{obs})}{\sqrt{2}}\right),$$

where $\text{erfc}(x) = 1 - \text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt$.

2.5.4 Tests Based on Random Walks III

Frequency Test in Blocks

Split $n = mr$ bits into r blocks of length m . For each block i :

$$Z_i = \frac{S_m^{(i)}}{\sqrt{m}} = \frac{1}{\sqrt{m}} \sum_{j=1}^m (2B_{(i-1)m+j} - 1).$$

Statistic: $T(\text{obs}) = \frac{1}{m} \sum_{i=1}^r (S_m^{(i)})^2 \approx \chi_r^2.$

2.5.4 Tests Based on Random Walks IV

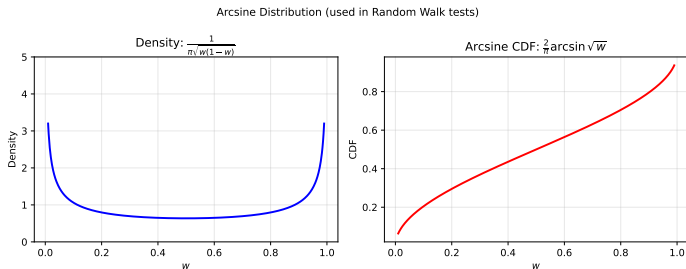
Arcsine Law Based Test

Let L_{2r}^+ = number of steps k where $S_k > 0$ (or $S_{k-1} > 0$) in a $2r$ -step walk. **Arcsine law:**

$$\lim_{r \rightarrow \infty} \mathbb{P}\left(\frac{L_{2r}^+}{2r} \leq w\right) = \frac{2}{\pi} \arcsin \sqrt{w}, \quad w \in [0, 1].$$

The density $\frac{2}{\pi} \arcsin \sqrt{w}$ is U-shaped: the walk spends most of its time *either* above or below zero.

2.5.4 Tests Based on Random Walks V



Test: $T(\text{obs}) = \frac{1}{n} \sum_{k=1}^n d_k$ where $d_k = \mathbb{I}(S_k > 0 \vee S_{k-1} > 0)$. p -value: $p \approx 1 - \frac{2}{\pi} \arcsin \sqrt{T(\text{obs})}$.

2.5.4 Tests Based on Random Walks VI

Random Excursion Test

Let J_n = number of zeros (returns to 0) among S_k , $k = 1, \dots, n$. Under $\mathcal{H}_0$:

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\frac{J_n}{\sqrt{n}} \leq t\right) = \sqrt{\frac{2}{\pi}} \int_0^t e^{-x^2/2} dx.$$

Reject if $J_n(\text{obs}) \leq 0.008\sqrt{n}$ (NIST recommendation).

Distribution of visits per excursion: Let $L(x)$ = number of visits to x during the 0th excursion.

$$\mathbb{P}(L(x) = k) = \begin{cases} 1 - \frac{1}{2|x|}, & k = 0, \\ \frac{1}{4x^2} \left(1 - \frac{1}{2|x|}\right)^{k-1}, & k \geq 1. \end{cases} \quad (2.5.22)$$

2.5.5 Permutation Tests I

Derangement Permutation Test:

Definition: Derangement

A *derangement* of $[m] = \{1, \dots, m\}$ is a permutation σ with *no fixed points*: $\sigma(i) \neq i$ for all i .

Lemma 2.5.10

The number of derangements of $[m]$ is:

$$d_m = \sum_{k=0}^m (-1)^k \binom{m}{k} (m-k)! = m! \sum_{k=2}^m (-1)^k \frac{1}{k!}.$$

As $m \rightarrow \infty$: $d_m/m! \rightarrow e^{-1} \approx 0.368$.

2.5.5 Permutation Tests II

Algorithm 3: Derangement Permutation Test

- ① $\xi = 0$
- ② **for** $i = 1$ to r :
- ③ $\sigma = \text{Fisher-Yates}(m, (u_{(i-1)m+1}, \dots, u_{im}))$
- ④ **if** σ is a derangement: $\xi = \xi + 1$
- ⑤ **Compute:**

$$T(\text{obs}) = \frac{|\xi - re^{-1}|}{\sqrt{re^{-1}(1 - e^{-1})}}, \quad p = \text{erfc}\left(\frac{T(\text{obs})}{\sqrt{2}}\right).$$

Under $\mathcal{H}_0$: $\mathbb{P}(\xi_i = 1) = e^{-1}$, so ξ is binomial; CLT applies.

2.5.5 Permutation Tests III

Fixed Point Permutation Test:

For each m -tuple $\mathbf{u}_i$, generate permutation σ^i and count $\xi_i = \#\{j : \sigma^i(j) = j\}$ (fixed points).

Under $\mathcal{H}_0$: $\xi_i \xrightarrow{\mathcal{D}} N$ (Poisson with mean 1) as $m \rightarrow \infty$.

Use boxes $A_0 = \{0\}$, $A_1 = \{1\}$, $\dots$, $A_9 = \{9\}$, $A_{10} = \{10, 11, \dots\}$. Expected probabilities:

$p_j = e^{-1}/j!$ for $j = 0, \dots, 9$, $p_{10} = 1 - \sum_{j=0}^9 p_j$.

Apply chi-square test with 10 degrees of freedom.

2.5.5 Python Code: Permutation Tests

```
import numpy as np
from scipy import stats

def fisher_yates(m, u):
    """Generate permutation of {0,...,m-1} from u in [0,1)^m."""
    S = list(range(m))
    for i in range(m, 0, -1):
        k = int(i * u[i-1])    # k in {0,...,i-1}
        S[k], S[i-1] = S[i-1], S[k]
    return S

def is_derangement(sigma):
    """True if sigma has no fixed points."""
    return all(sigma[i] != i for i in range(len(sigma)))

rng = np.random.default_rng(42)
m, r = 10, 1000    # m-tuples, r repetitions
n = m * r
u = rng.random(n)

xi = 0
for i in range(r):
    ui = u[i*m:(i+1)*m]
    sigma = fisher_yates(m, ui)
    if is_derangement(sigma):
        xi += 1
```

2.5.6 p-Values and Second-Level Testing I

Key insight: Under $\mathcal{H}_0$ with a *continuous* statistic T , the p -value is itself $\mathcal{U}(0, 1)$:

$$p = 1 - F(T(\text{obs})).$$

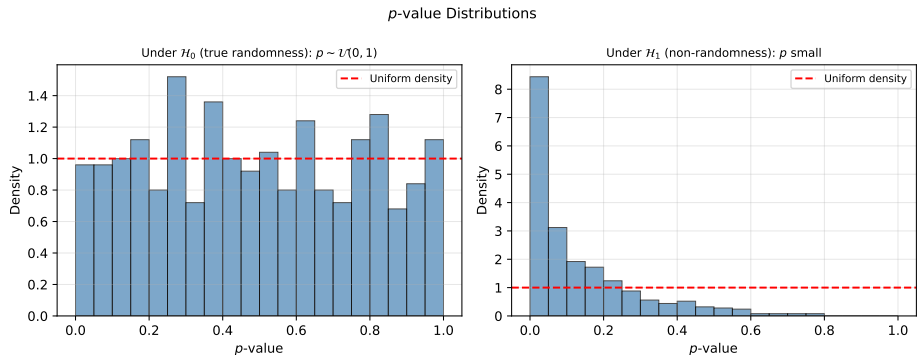
Proof sketch

$$\mathbb{P}(p \leq t) = \mathbb{P}(1 - F(T) \leq t) = \mathbb{P}(T \geq F^{-1}(1 - t)) = 1 - F(F^{-1}(1 - t)) = t.$$

Second-level testing procedure:

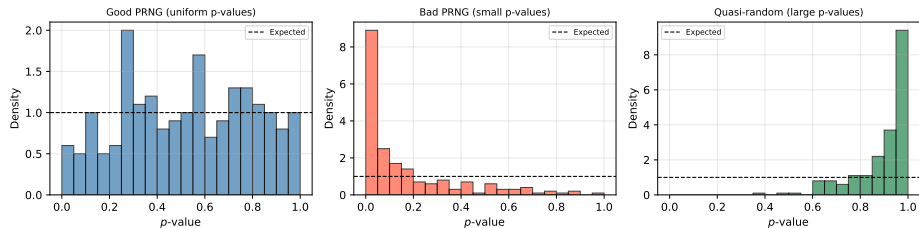
- ① Split an Rn -length sequence into R subsequences of length n
- ② For each subsequence i : compute statistic $T^{(i)}$ and $p^{(i)} = 1 - F(T^{(i)})$
- ③ Test uniformity of $p^{(1)}, \dots, p^{(R)}$ via chi-square:
 - Partition $[0, 1)$ into s equal intervals: $P_i = [(i-1)/s, i/s)$
 - $O_i = |\{j : p^{(j)} \in P_i\}|$, $E_i = R/s$
 - $X_{\text{final}}^2 = \sum_{i=1}^s (O_i - E_i)^2 / E_i \approx \chi_{s-1}^2$

2.5.6 p-Values and Second-Level Testing II



2.5.6 p-Values and Second-Level Testing III

Second-Level Testing: Histograms of $R = 200$ p -values



2.5.6 p-Values and Second-Level Testing IV

Practical recommendations (NIST):

- Use $1000 \leq R \leq 10000$ sequences
- Sequence length $2^{20} \leq n \leq 2^{32}$
- Significance level $\alpha \in [1/1000, 1/100]$
- A PRNG passes if $\hat{B} \in [1 - \alpha \pm 3\sqrt{\alpha(1 - \alpha)/R}]$ (fraction of sequences with $p > \alpha$)

Example 2.5.12: Second-level KS and chi-square tests on sets A, B, C with $R = 200$ sequences of $n = 50$.

- Set A: p -values uniform — passes
- Set B: p -values concentrated near 0 — **fails**
- Set C (quasi-random): p -values concentrated near 1 — **detected as non-random!**

Second-level testing *detects* the “too perfect” uniformity of quasi-random sequences.

2.5.6 Python Code: Second-Level Testing

```
import numpy as np
from scipy import stats

def second_level_test(sequences, test_fn, s=10):
    """
    sequences: list of 1D arrays, each a subsequence
    test_fn: function returning p-value for a sequence
    s: number of bins for chi-square
    """
    p_vals = [test_fn(seq) for seq in sequences]
    # Chi-square uniformity test on p-values
    bins = np.linspace(0, 1, s+1)
    obs, _ = np.histogram(p_vals, bins=bins)
    exp = [len(p_vals) / s] * s
    chi2, p_final = stats.chisquare(obs, exp)
    return p_vals, chi2, p_final

# Generate R=200 sequences of n=50 from PCG64 (Set A)
rng = np.random.default_rng(0)
R, n = 200, 50
sequences_A = [rng.random(n) for _ in range(R)]

def ks_pvalue(seq):
    _, p = stats.kstest(seq, 'uniform')
    return p

p_vals, chi2, p_final = second_level_test(sequences_A, ks_pvalue)
```

2.6 Bibliographical Notes I

Foundational references:

- **Knuth** [107]: “The Art of Computer Programming, Vol. 2” — comprehensive treatment of LCGs, period theory, and statistical tests
- **L’Ecuyer** [118, 122]: Theoretical and practical PRNG design; combined generators and TestU01 test suite
- **Marsaglia** [141, 145]: Diehard test battery; birthday spacing test; spectral tests
- **NIST SP 800-22** [187]: Statistical test suite for cryptographic PRNGs; 15 standardized tests

Modern generators:

- **PCG64** (O’Neill [162]): Permuted Congruential Generators — current NumPy default
- **MT19937** (Matsumoto & Nishimura [146]): Mersenne Twister; period $2^{19937} - 1$; passes Diehard but fails some TestU01 tests
- **AES-CTR, ChaCha20**: Cryptographically secure stream ciphers

2.6 Bibliographical Notes II

Shuffling and card mixing:

- **Bayer & Diaconis** [14]: “Trailing the Dovetail Shuffle to its Lair” — ~ 7 riffle shuffles needed for $n = 52$ cards
- **Aldous & Diaconis** [2]: Markov chain coupling methods for mixing time analysis

Theoretical test results:

- **Weiss** [210]: Normal approximation for empty boxes ($\mathcal{E}_{k,r}$)
- **Holst** [83], **L’Ecuyer et al.** [122]: Known results on the serial test chi-square statistic
- **Tsang et al.** [205]: Random excursion test analysis
- **L’Ecuyer & Simard** [169, 170]: Reliable test design using Berry-Esseen bounds

2.7 Exercises: Selected Problems I

Theoretical exercises (sampling):

2.T.1 (Markov and Chebyshev inequalities):

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}X}{a}, \quad \mathbb{P}(|X - \mathbb{E}X| \geq b) \leq \frac{\text{Var}X}{b^2}.$$

2.T.4: If $U_1, \dots$ i.i.d. $\mathcal{U}(0, 1)$ and $M \geq 2$, then $\lfloor MU_i \rfloor$ is a random sequence from $\bar{M}$.

2.T.8: Verify Fisher-Yates generates a uniform random permutation (by induction on the swap steps).

2.T.17 (Serial test): Show $\mathbb{E}X_k^2 = k - 1$ for the serial test statistic.

2.7 Exercises: Selected Problems II

Lab exercises (sampling):

2.L.1: Implement LCG with $M = 2^{11} - 1$, $a = 5$, $c = 1$. Generate 10^4 numbers, plot (i, u_i) pairs. Find the period. Check Theorem 2.4.1.

2.L.2: Compare Fisher-Yates vs. modified Algorithm B (where line 3 uses $k = \text{random}(1, \dots, n)$ instead of $\{1, \dots, i\}$). Test with fixed-point permutation test.

2.L.3: Generate $n = 10^5$ numbers with MT19937 and with LCG ($M = 2^{31} - 1$, $a = 16807$). Compute the Pearson autocorrelation between $(y_1, \dots, y_{n-1})$ and $(y_2, \dots, y_n)$.

2.L.5: Quasi-random test: $u_i = i/M \bmod 1$ with $M = 100$, $i = 1, \dots, 10^6$. Perform KS, chi-square, and second-level tests.

Chapter 2: Summary

Key concepts covered:

- PRNG definition (5-tuple (S, s_0, f, V, g))
- Random sequences: $\mathcal{U}(0, 1)$, bits, integers
- Fisher-Yates shuffle; shuffling cards
- Python: `default_rng`, PCG64, MT19937
- RC4 stream cipher (KSA + PRGA)
- LCG period conditions (Theorem 2.4.1)
- Bit generators: LFSR, XORG, SXR, BBS

Statistical testing:

- KS test: $D_n = \sup |\hat{F}_n - t|$
- Chi-square: $X^2 = \sum (O_i - E_i)^2 / E_i$
- Balls-and-boxes: collision test, birthday spacing, poker test
- Random walk tests: monobit, arcsine law, excursion test
- Permutation tests: derangement, fixed-point
- Second-level testing: detect “too perfect” sequences

Take-home message

No PRNG is perfect. Modern generators (PCG64) pass all practical tests, but cryptographic applications require additional guarantees (AES-CTR, ChaCha20). Always use second-level testing when verifying PRNG quality.